

- Description

This module manages a smart solar inverter system, ensuring that:

1. The system continuously monitors the current DC voltage from the solar panels (in volts).
2. If the voltage falls below a predefined minimum threshold, the inverter output is increased by a fixed percentage to compensate for low generation.
3. If the voltage exceeds a predefined maximum threshold, the inverter output is decreased by a fixed percentage to protect the system.
4. If the voltage is within the optimal range, the inverter output remains unchanged.

This system helps optimize power conversion efficiency and ensures safe operation of the solar power system.

- Specification

module SmartSolarInverterControl

var currentVoltage, minVoltage, maxVoltage, inverterOutput: real

process updateVoltage(newVoltage: real) returns (currentVoltage: real, inverterOutput: real)

functional scenario form:

currentVoltage < minVoltage && inverterOutput = inverterOutput * 1.1, output "ALERT: Voltage too low (" + currentVoltage + " V). Inverter output increased to " + inverterOutput + " V.", Inverter output increased.

||

currentVoltage > maxVoltage && inverterOutput = inverterOutput * 0.9, output "ALERT: Voltage too high (" + currentVoltage + " V). Inverter output decreased to " + inverterOutput + " V.", Inverter output decreased.

||

currentVoltage >= minVoltage && output "Voltage is optimal (" + currentVoltage + " V). Inverter output remains at " + inverterOutput + " V.", No adjustment required.

end process

end module

- Java Code

```
package code;
public class c107_SmartSolarInverter {
    private double minVoltage;
    private double maxVoltage;
    private double currentVoltage;
    private double inverterOutput;
    public c107_SmartSolarInverter(double minVoltage, double maxVoltage
, double initialValue, double initialOutput) {
        this.minVoltage = minVoltage;
        this.maxVoltage = maxVoltage;
```

```

        this.currentVoltage = initialVoltage;
        this.inverterOutput = initialOutput;
    }
    public void updateVoltage(double newVoltage) {
        currentVoltage = newVoltage;
        adjustInverterOutput();
        System.out.printf("Current Panel Voltage: %.2f V, Inverter Output: %.2f V\n", currentVoltage, inverterOutput);
        if (currentVoltage < minVoltage) {
            System.out.println("ALERT: Voltage too low. Increasing inverter output.");
        } else if (currentVoltage > maxVoltage) {
            System.out.println("ALERT: Voltage too high. Decreasing inverter output.");
        } else {
            System.out.println("Voltage is optimal. No adjustment required.");
        }
    }
    private void adjustInverterOutput() {
        if (currentVoltage < minVoltage) {
            inverterOutput = inverterOutput * 1.1;
        } else if (currentVoltage > maxVoltage) {
            inverterOutput = inverterOutput * 0.9;
        }
    }
    public double getInverterOutput() {
        return inverterOutput;
    }
    public static void main(String[] args) {
        c107_SmartSolarInverter inverter = new c107_SmartSolarInverter(300.0, 400.0, 350.0, 350.0);
        // testcase-VT:
        //     inverter.updateVoltage(280.0);
        //     inverter.updateVoltage(360.0);
        //     inverter.updateVoltage(420.0);

        // testcase-FT:
        //     inverter.updateVoltage(-4.1527529154101297E298);
        //     inverter.updateVoltage(-1.7976931348623157E308);
        //     inverter.updateVoltage(0.0);

        // testcase-Z3:
        //     inverter.updateVoltage(280.0);

```

```
//      inverter.updateVoltage(360.0);
//      inverter.updateVoltage(420.0);

// testcase-UVT:
inverter.updateVoltage(280.0);
inverter.updateVoltage(360.0);
inverter.updateVoltage(420.0);
}
}
```

- Coverage report

1. Test case number: 3
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c107_SmartSolarInverter.java	100.0 %	8	0	8
▼ c107_SmartSolarInverter	100.0 %	8	0	8
● updateVoltage(double)	100.0 %	4	0	4
■ adjustInverterOutput()	100.0 %	4	0	4
● c107_SmartSolarInverter(double)		0	0	0
● main(String[])		0	0	0

3. Vibration testing instruction coverage: 97.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	106	21,859	21,965
▼ src	0.5 %	106	21,859	21,965
▼ code	0.5 %	106	21,859	21,965
▼ c107_SmartSolarInverter.java	97.2 %	106	3	109
▼ c107_SmartSolarInverter	97.2 %	106	3	109
● updateVoltage(double)	100.0 %	47	0	47
■ adjustInverterOutput()	100.0 %	26	0	26
● c107_SmartSolarInverter(double)	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

4. Fuzz testing branch coverage: 25%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.2 %	2	840	842
▼ src	0.2 %	2	840	842
▼ code	0.2 %	2	840	842
▼ c107_SmartSolarInverter.java	25.0 %	2	6	8
▼ c107_SmartSolarInverter	25.0 %	2	6	8
● updateVoltage(double)	25.0 %	1	3	4
■ adjustInverterOutput()	25.0 %	1	3	4
● c107_SmartSolarInverter(double)		0	0	0
● main(String[])		0	0	0

5. Fuzz testing instruction coverage: 74.3%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.4 %	81	21,884	21,965
▼ src	0.4 %	81	21,884	21,965
▼ code	0.4 %	81	21,884	21,965
▼ c107_SmartSolarInverter.java	74.3 %	81	28	109
▼ c107_SmartSolarInverter	74.3 %	81	28	109
● updateVoltage(double)	72.3 %	34	13	47
■ adjustInverterOutput()	53.8 %	14	12	26
● c107_SmartSolarInverter(double)	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c107_SmartSolarInverter.java	100.0 %	8	0	8
▼ c107_SmartSolarInverter	100.0 %	8	0	8
● updateVoltage(double)	100.0 %	4	0	4
■ adjustInverterOutput()	100.0 %	4	0	4
● c107_SmartSolarInverter(double)		0	0	0
● main(String[])		0	0	0

7. Z3 instruction coverage: 97.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	106	21,859	21,965
▼ src	0.5 %	106	21,859	21,965
▼ code	0.5 %	106	21,859	21,965
▼ c107_SmartSolarInverter.java	97.2 %	106	3	109
▼ c107_SmartSolarInverter	97.2 %	106	3	109
● updateVoltage(double)	100.0 %	47	0	47
■ adjustInverterOutput()	100.0 %	26	0	26
● c107_SmartSolarInverter(double)	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.0 %	8	834	842
▼ src	1.0 %	8	834	842
▼ code	1.0 %	8	834	842
▼ c107_SmartSolarInverter.java	100.0 %	8	0	8
▼ c107_SmartSolarInverter	100.0 %	8	0	8
● updateVoltage(double)	100.0 %	4	0	4
■ adjustInverterOutput()	100.0 %	4	0	4
● c107_SmartSolarInverter(double)		0	0	0
● main(String[])		0	0	0

9. Unimproved V-method instruction coverage: 97.2%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.5 %	106	21,859	21,965
▼ src	0.5 %	106	21,859	21,965
▼ code	0.5 %	106	21,859	21,965
▼ c107_SmartSolarInverter.java	97.2 %	106	3	109
▼ c107_SmartSolarInverter	97.2 %	106	3	109
● updateVoltage(double)	100.0 %	47	0	47
■ adjustInverterOutput()	100.0 %	26	0	26
● c107_SmartSolarInverter(double)	100.0 %	15	0	15
● main(String[])	100.0 %	18	0	18